

4-9 某系统进程调度状态变迁图如图 4.31 所示，请说明：

(1) 什么原因会导致发生变迁 2、变迁 3、变迁 4 ？

2：时间片到

3：服务请求（请求 I/O 等）

4：服务完成/事件来到

(2) 在什么情况下，一个进程的变迁 3 能立即引起另一个进程发生变迁 1 ？

就绪队列中至少有一个可运行进程，调度程序从就绪队列中选取某个就绪进程使其运行

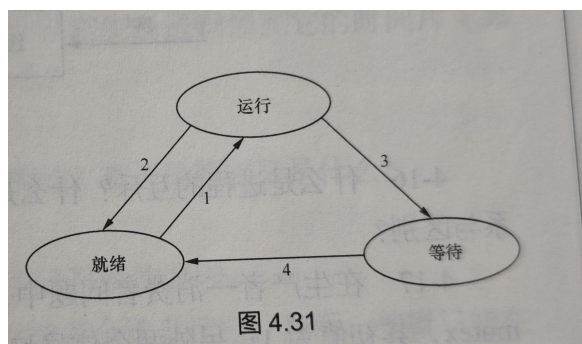
(3) 下列因果变迁是否可能发生？若可能，需要什么条件？

a. $2 \rightarrow 1$ ； b. $3 \rightarrow 2$ ； c. $4 \rightarrow 1$

A：会发生，不需要条件。当运行进程转为就绪后，调度程序就从就绪队列选取某个就绪进程使其运行

B：不可能。一个进程阻塞不会直接使另一个运行进程变为就绪

C：可能。若 CPU 空闲，就绪队列为空，则该就绪进程被立即调度运行。或者它的优先级足够高，会触发抢占 CPU



4-14 如图 4.33 所示的进程流图中，有 5 个进程合作完成某一任务。说明这 5 个进程之间的同步关系，并用 P,V 操作实现，要求写出程序描述

p1 结束，p2,p3,p4 才能执行

P3,p4 结束,p5 才能执行

这里用两个信号量分别控制上面两个关系，也可以用多个信号量实现。

```
main()
```

```
{
```

```
    sem s1=0;
```

```
    sem s2=0;
```

```
    cobegin
```

```
        p1();
```

```
        p2();
```

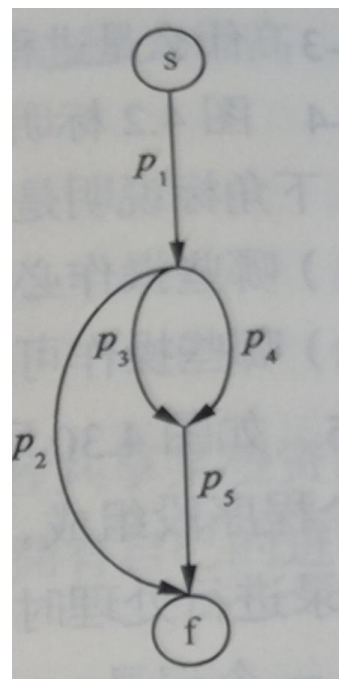
```
        p3();
```

```
        p4();
```

```
        p5();
```

```
    coend
```

```
}
```



P1()

```
{
```

```
    V(s1);
```

```
    V(s1);
```

```
    V(s1); //允许 p2,p3,p4 启动
```

```
}
```

P2()

```
{
```

```
    P(s1);
```

```
    ...//p2 工作
```

```
}
```

P3()

```
{
```

```
    P(s1);
```

```
    ...//p3 工作
```

```
    V(s2);
```

```
}
```

P4()

```
{
```

```
    P(s1);
```

```
    ...//p4 工作
```

```
    V(s2);
```

P5()

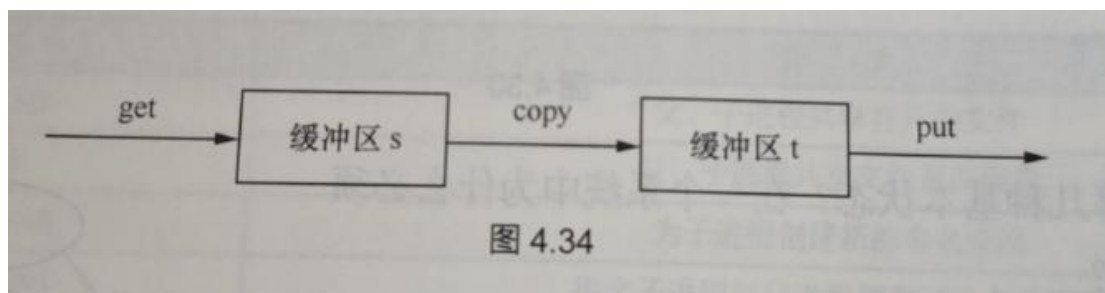
```
{
```

```
    P(s2);
```

```
    P(s2); //s2 达到 2 后 p5 才能执行
```

```
    ...//p5 工作
```

4-15 如图 4.34 所示, `get`、`copy` 和 `put` 三个进程共用两个缓冲区 `s` 和 `t` (其大小每次存放一个记录), `get` 进程负责不断地把输入记录输入缓冲区 `s` 中, `copy` 进程负责从缓冲区 `s` 中取出记录复制到缓冲区 `t` 中, 而 `put` 进程负责从缓冲区 `t` 中取出记录打印。试用 PV 操作实现这三个进程之间的同步, 并写出程序描述。



```
main()
{
    // 对于缓冲区 s
    sem s_a = 0; // s 中是否有可供 copy 取走的数据 (初始无)
    sem s_b = 1; // s 中是否有空位可放 (初始有)
    // 对于缓冲区 t
    sem t_a = 0; // t 中是否有可供 put 取走的数据 (初始无)
    sem t_b = 1; // t 中是否有空位可放 (初始有)
    cobegin
        get();
        copy();
        put();
    coend
}
// get: 产生数据放入 s
get(){
    while (true) {
        P(s_b); // 等 s 有空位
        ... // 把输入记录输入缓冲区 s
        V(s_a); // 告知 s 中有数据
    }
}
// copy: 从 s 取, 放到 t
copy()
{
    while (true) {
        P(s_a); // 等 s 有数据
        ... // 从 s 中复制数据
        V(s_b); // 释放 s 的空位
    }
}
```

```

    P(t_b);                // 等 t 有空位
    ...                    // 把记录输入缓冲区 t
    V(t_a);                // 告知 t 中有数据
}
}
// put: 从 t 取并打印
put()
{
    while (true) {
        P(t_a);            // 等 t 有数据
        ...                // 从 t 取走数据
        V(t_b);            // 释放 t 的空位
        print(y);          // 打印
    }
}

```

4-18 判断下面同步算法是否有错，若有错，请指出错误原因并改正。

(1) 3 个进程并发活动的进程流图如图 4-35 所示，其同步算法描述如下：

```

main( )
{
    int s = -1;
    cobegin
        p1( );
        p2( );
        p3( );
    coend
}

p1( )      p2( )      p3( )
{          {          {
    :          :          :
    v(s);      v(s);      p(s);
    :          :          :
}          }          }

```

错误，信号灯初值不能为负

```

main( )
{
    int s13 = 0; // 用于 p1 唤醒 p3 的信号量
    int s23 = 0; // 用于 p2 唤醒 p3 的信号量
    cobegin
        p1( );
        p2( );
        p3( );
    coend
}

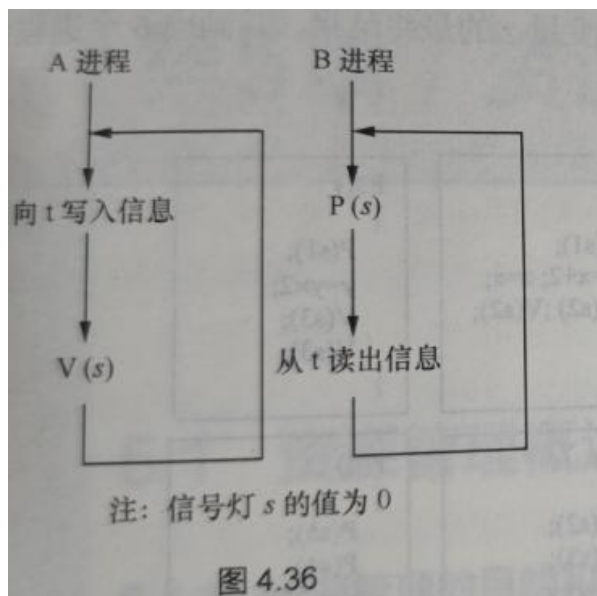
```

```

p1( )
{
    // p1 的业务逻辑
    :
    v(s13); // p1 完成后, 向p3 发送信号 (s13+1)
}
p2( )
{
    // p2 的业务逻辑
    :
    v(s23); // p2 完成后, 向p3 发送信号 (s23+1)
}
p3( )
{
    p(s13); // 等待p1 的信号 (s13 从0→-1, 阻塞, 直到p1 执行v)
    p(s23); // 等待p2 的信号 (s23 从0→-1, 阻塞, 直到p2 执行v)
    // p3 的业务逻辑 (仅在p1 和p2 都完成后执行)
    :
}

```

(2) 设 a, b 两进程共用一个缓冲区 t, a 向 t 写入信息, b 从 t 读出信息, 算法框图如图 4-36 所示。



错误, 一个信号量无法实现正确逻辑, A 进程在写入前没有等待缓冲区为空, 若写入过快, 会导致上一条数据被覆盖, 无法保证写后读的顺序。

正确做法是使用两个信号量

sa: 表示缓冲区中是否有可供打印的计算结果, 其初值为 0。

sb: 表示缓冲区有无空位置存放新的信息, 其初值为 1。

A 进程:

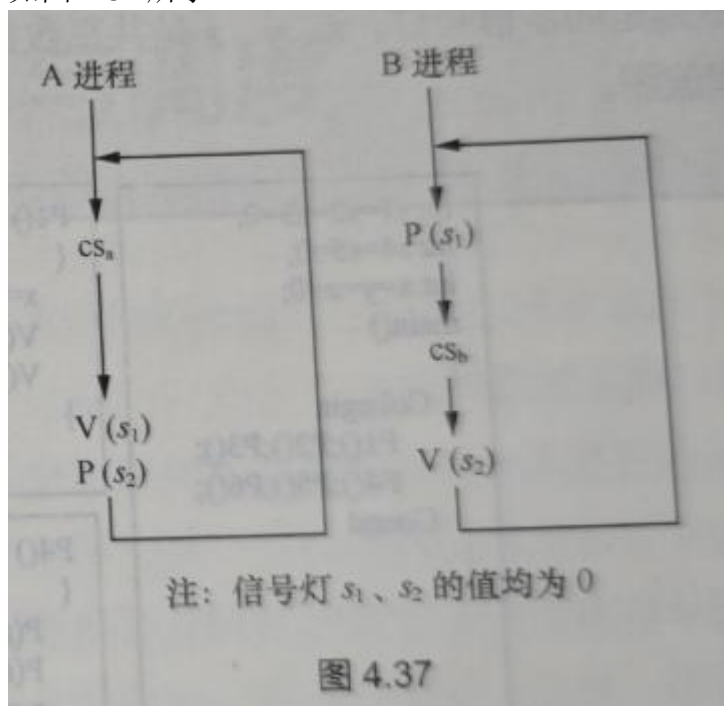
```
while (true) {
```

```

    P(sb);          // 等待缓冲区为空
    write(t);        // 向 t 写入信息
    V(sa);          // 缓冲区有数据可读
}
B 进程:
while (true) {
    P(sa);          // 等待缓冲区有数据
    read(t);        // 从 t 读取信息
    V(sb);          // 缓冲区重新变为空
}

```

(3) 设 a, b 为两个并发进程，它们共享一临界资源。其执行临界区的算法框图如图 4-37 所示。



错误，不需要使用两个信号量。它强加了 A-B-A-B... 的交替顺序，不满足通用的互斥进入要求。

```

main()
{
    int mutex=1;    // 互斥信号灯
    cobegin
        p_a();
        p_b();
    coend
}
p_a()
{
    :
    p(mutex);
}

```

```

    CSa;
    v(mutex);
    :
}
pβ()
{
    :
    p(mutex);
    CSβ;
    v(mutex);
    :
}

```

4-30 某工厂有两条生产线、一条组装线和两条传输线。两条生产线分别生产 A、B 两种零件；组装线将 A、B 两个零件组装成一个产品编号；传输线负责产品的传送。生产线上每生产一个零件，需将零件送到货架 F1 或 F2 上，其中，F1 存放零件 A，F2 存放零件 B，货架一次只能存放一个零件。组装线每次从货架上取一个 A 零件和一个 B 零件，组装成一个产品并按序给产品编号，然后将已编号的产品放入货架 F3，F3 一次只能存放一个产品。传输线分别负责传送不同类型编号的产品，其中，1 号传输线负责传送偶数编号的产品，2 号传输线负责传送奇数编号的产品。请用 P、V 操作和信号灯进行正常管理。要求：

(1) 说明信号灯的初值及语义。

```
int ID = 1; // 产品编号
```

信号灯:

```

sem empty_F1 = 1; //货架 F1 是否为空，初值为 1。
sem full_F1 = 0; //货架 F1 是否已满，初值为 0。
sem empty_F2 = 1; //货架 F2 是否为空，初值为 1。
sem full_F2 = 0; //货架 F2 是否已满，初值为 0。
sem empty_F3 = 1; //货架 F3 是否为空，初值为 1
sem full_F3_even = 0; //货架 F3 是否有偶数编号产品，初值为 0。
sem full_F3_odd = 0; //货架 F3 是否有奇数编号产品，初值为 0。

```

(2) 写出同步算法来描述工厂的生产活动。

```
main()
```

```

{
    cobegin
    {
        Produce_A();    // 生产线 A
        Produce_B();    // 生产线 B
        Assemble();     // 组装线
        Transfer_1();   // 1 号传输线 (偶数)
        Transfer_2();   // 2 号传输线 (奇数)
    }
    coend
}
// 生产线 A 进程
Produce_A()
{
    while(true)
    {
        ...// 生产一个 A 零件
        P(empty_F1);    // 等待 F1 货架变空
        ...// 将 A 零件放入 F1
        V(full_F1);     // 通知 F1 货架满了 (有 A 零件了)
    }
}
// 生产线 B 进程
Produce_B()
{
    while(true)
    {
        ...// 生产一个 B 零件
        P(empty_F2);    // 等待 F2 货架变空
        ...// 将 B 零件放入 F2
        V(full_F2);     // 通知 F2 货架满了 (有 B 零件了)
    }
}
// 组装线进程
Assemble()
{
    int current_ID; // 存储当前产品编号
    while(true)
    {
        P(full_F1);     // 等待 F1 上有 A 零件
        P(full_F2);     // 等待 F2 上有 B 零件

        ...// 从 F1 取走 A 零件
        V(empty_F1);    // F1 货架变空
    }
}

```



```

...// 从 F2 取走 B 零件
V(empty_F2);    // F2 货架变空
...// 组装产品

current_ID = ID; // 获取当前编号
ID++;           // 编号递增
...// 将 current_ID 赋给产品

P(empty_F3);    // 等待 F3 货架变空
...// 将编号的产品放入 F3

// 根据产品编号, 唤醒对应的传输线
if (current_ID % 2 == 0)
{
    V(full_F3_even); // F3 有一个偶数产品
}
else
{
    V(full_F3_odd);  // F3 有一个奇数产品
}
}
}
// 1 号传输线进程 (负责偶数)
Transfer_1()
{
    while(true)
    {
        P(full_F3_even); // 等待 F3 上出现偶数产品
        ...// 从 F3 取走偶数产品
        V(empty_F3);     // F3 货架变空
        ...// 传送该偶数产品
    }
}
// 2 号传输线进程 (负责奇数)
Transfer_2()
{
    while(true)
    {
        P(full_F3_odd);  // 等待 F3 上出现奇数产品
        ...// 从 F3 取走奇数产品
        V(empty_F3);     // F3 货架变空
        ...// 传送该奇数产品
    }
}

```

}